

Pramaan — Evidence Pipeline & Team Plan

What goes in, what happens to it, and who finishes what

What goes into the app, what happens to it, and who finishes what. One rule above everything else in this document: **every feature that ends up in the demo has to actually work, on real evidence, live.** No placeholder screens, no "this would work if," no pre-recorded fallback video. If something can't be made real in time, it comes out of the demo — it doesn't get faked.

Design ground rule still applies. Light theme only. DESIGN.md is dead.

1. What goes in, what happens to it, and why

1.1 What file do you actually give the app?

There are two kinds of input. They are not equivalent, and the app treats them differently.

A. A forensic disk image — the real input. This is a copy of the entire storage inside the DVR/NVR: .dd, .raw, .E01, or .ex01. Not a video file — a copy of the whole disk, byte for byte, the same way you'd image a laptop hard drive for a forensic case. This is what the problem statement actually asks for ("create forensic images"), and it's the only input type where the full pipeline — identify, parse, recover deleted footage — applies, because all of that only makes sense if you have the recorder's whole filesystem, not just a clip cut out of it.

What's actually inside a disk image like this:

- The recorder's own filesystem/index — a table the recorder itself maintains, listing every recording it knows about: which channel (camera), start time, end time, and whether that slot is still valid or has been marked for reuse ("deleted"). Different vendors store this differently (Hikvision's HIKBTREE, Dahua's DHFS), but every vendor has something like it.
- The actual video data for every recording still physically present on the disk — including recordings the index says are "deleted." DVRs work like a circular buffer: when disk space runs out, the oldest recording's index entry gets marked as reusable and new footage gets written over it eventually. Until that overwrite actually happens, the old video bytes are still sitting on the disk, just no longer listed as valid. That's what "recovering deleted footage" means here — reading data that's still physically there but that the recorder itself no longer shows you.
- Sometimes: system event logs (e.g. Hikvision keeps a separate log region with motion/alarm events), which is extra metadata beyond just the recording list.

B. An already-exported video clip — the shortcut path. .mp4, .avi, .dav, .264, etc. This is what you get if someone already pressed "export" on the DVR itself, or handed you a clip pulled off the device some other way. It contains only the video frames for that one clip — no index, no deleted recordings, because those only exist on the full disk. The app still accepts this (hash it, analyze it, run it through cross-camera trace), but it skips identification and recovery entirely, because there's no filesystem here to identify or recover from. This path is a convenience, not the graded forensic workflow — don't present it as the same thing in a demo.

1.2 Metadata comes out first, before anything is touched

This is the part that was confusing, so here it is plainly: **the app reads metadata straight from the original disk image, before it creates any new file.** It does not extract metadata, then convert the video, then somehow lose or reconstruct that metadata. The order is:

1. Read the recorder's index directly off the disk image (still just reading, nothing written or converted yet). This gives channel number, start/end time, and deleted-or-not — for every recording, without decoding a single frame of video.
2. For one specific recording, go to its byte range on the disk image and pull out the video data sitting there.
3. Only now, for that one recording, strip the vendor's wrapper and write a new, ordinary .mp4 file so it can actually

be played and analyzed.

Steps 1 and 2 never modify or discard anything — they're reading the same untouched disk image the whole time. Step 3 is the only place a new file gets created, and by then the metadata that mattered (channel, timestamps, deleted status, and the hash of the original image) has already been pulled out and saved in the case database, tied permanently to that recording. The new .mp4 file doesn't need to carry that metadata in its own header — an ordinary video file format has no field for "recorder allocation state" anyway. The database is where that metadata lives; the .mp4 is just something a person or an AI model can actually open.

So why make the new file at all, if the metadata's already safe? Because nothing else can use the recorder's raw video data directly. A vendor's on-disk video block is wrapped in that vendor's own private format — no ordinary video player can open it, and the app's own AI analysis (which decodes frames with OpenCV) can't read it either. Producing one ordinary .mp4 is what lets a human watch the footage and lets the AI pipeline analyze it. It's a convenience copy for consumption, not a replacement for the original evidence — the original disk image stays untouched and is still the thing that's hashed, cited, and treated as evidence in the report.

The only twist: turning the vendor's wrapped video into a normal .mp4 should never re-encode the actual video (decode it and compress it again) if it can be avoided, because re-encoding is lossy and slow. It should copy the underlying video bytes as-is into a new container — same picture, same quality, just repackaged. Re-encoding is a last resort, only when the data is too damaged to copy cleanly, and it must be logged so nobody mistakes it for a clean copy.

1.3 Walkthrough — a seized NVR, start to finish

A shop is robbed. Police seize the shop's NVR (the recorder box) and hand the examiner its hard drive.

1. **Acquire.** The examiner connects the drive through a write-blocker and images it: every byte, copied into one file, `shopcase_nvr.E01`. Pramaan hashes this file (MD5 + SHA-256) the moment it's registered. This file is never modified again for the rest of the case — every later step reads from it, nothing writes back to it.
2. **Identify.** Pramaan looks at the first chunk of that image, finds the signature that says "this is a Hikvision recorder, HIKBTREE filesystem," and reports that with a confidence score, not a guess dressed up as certainty.
3. **Parse.** Pramaan walks the HIKBTREE index inside that same image (still hasn't created a single new file) and produces a list: Channel 1, 09:00–09:45, valid. Channel 1, 09:45–10:15, marked deleted. Channel 2, 09:00–10:15, valid. This list is most of what an investigator actually wants to know, and it came from reading index bytes only — no video has been touched yet.
4. **Recover.** For the deleted 09:45–10:15 entry, Pramaan goes to that byte range in the image and checks whether the video data is still there. It is (nobody's recorded over it yet), so it gets carved out. If it had been partially overwritten, Pramaan would recover only the part that's genuinely intact and mark the rest as unrecoverable — never guess at the missing part.
5. **Unwrap and repackage.** That carved-out data is still wrapped in Hikvision's own format. Pramaan strips that wrapper, finds the H.264 video underneath, and copies it — not re-encodes it — into an ordinary file: `channel1_0945-1015_deleted.mp4`. This is a new file. The original `shopcase_nvr.E01` is untouched.
6. **Hash the new file too.** This .mp4 gets its own hash, separate from the image's hash. The case record now says, in effect: this mp4 (hash X) came from `shopcase_nvr.E01` (hash Y), channel 1, byte offset Z, recorder said 09:45–10:15, the index had marked it deleted. Every fact traces back to where it actually came from.
7. **Normalize the timestamp.** The recorder's clock might be off — DVRs left un-synced drift over time. If the examiner has calibrated that device's drift, the corrected time gets recorded alongside the recorder's original time (both are kept, not one overwriting the other).
8. **Now the file is usable.** The examiner can play `channel1_0945-1015_deleted.mp4` in any ordinary video player. Pramaan's own analysis can decode it frame by frame looking for people, objects, or motion. If footage from Channel 2 covers an overlapping time window, cross-camera trace can compare the two for the same person.
9. **Report.** The final PDF cites the image's hash, the recording's hash, the byte offset it came from, the recorder's original timestamp and the corrected one, and the fact that this specific recording was recovered from a deleted index entry. It doesn't just show a video — it shows exactly where every fact in that entry came from.

That's the whole pipeline. Everything else in the app — cross-camera trace, findings, the report — operates on the outputs of steps 6–9, never on the original image directly.

2. Three scope decisions

Corrupted footage. Recover what's genuinely there. Bytes that are missing or overwritten get marked PARTIAL — never filled in with a guess. This is already how the code behaves (`recovery_status`); it has to stay that way everywhere, including anything new that gets built.

Upscaling or "enhancing" video — deliberately out of scope. It's not in the problem statement, and it shouldn't be added. AI super-resolution invents pixel detail that was never actually captured — a real risk in a forensic tool, not a nice-to-have feature, because it's exactly the kind of thing that gets video evidence challenged in court. If a judge or teammate raises it: Pramaan is one tool in a forensic workstation, the same way commercial tools like DVR Examiner or UFS Explorer are — recovery, parsing, and hashing here; a dedicated video-enhancement tool is a separate job for a separate tool. That's a stronger answer than trying to build a "safe" upscaler.

Cross-camera trace runs on the derived clips, not the original image. Person matching and face matching need to decode video frames, and that only works on the ordinary .mp4 produced in step 6 above, not on the raw disk image. That's expected and fine — it's exactly the "one place lossy analysis legitimately happens" from section 1.2, and every match still cites back to which recording, and therefore which hash, it came from.

3. Where things stand — spot-checked directly against the code, not a full re-audit

Area	Status	How I know
Cross-camera trace, Findings proximity flags	Shipped	Built and merged this session (Navneet).
Live devices	Removed entirely	Confirmed out of PS scope; deleted frontend/backend/tests/CI job.
Hikvision parser (<code>hikvision_fs.py</code>)	Solid	Cites 2 peer-reviewed papers + 1 corroborating open-source project; tracks deleted as a distinct state, not a guess.
Hikvision recovery on the current lab image	Bug, reproduced today	<code>lab_hikvision_fs.img</code> identifies correctly (HIKBTREE match, 0.6 confidence) but recover returns 0 segments. Not yet root-caused.
Dahua recovery on the real .dav	Works	Ran it today end to end: acquire -> identify (0.98 confidence) -> recover -> 1 segment recovered -> playable.
Dahua DHFS parsing	Still carve-only	<code>dahua_dhfs.py</code> scans for the DHAV frame signature and treats a 4KB gap as "deleted." It never reads the DHFS index the way Hikvision's parser reads HIKBTREE.
Recovery page deleted column/filter	Present	<code>deletedOnly</code> toggle + allocation glyphs exist in <code>CaseRecoverPage.tsx</code> .
Ranged export (<code>-ss/-c</code> copy)	Verifies before trusting	Checks the result before accepting it, falls back to re-encode on failure — the old silent-empty-file bug looks fixed.
Report HTML preview	Still dark-themed	<code>reporting.py</code> still sets <code>background:#0a0e1a</code> — the one screen where the rejected dark UI is still live.
Job log	Still recovery-only	Only filters <code>job.kind === "recovery"</code> — analytics, export, and cross-camera jobs are invisible in it.
Settings	"Parser sanity check" panel still present	Still shown as a product feature; it's a test-suite runner and belongs in CI, not the app.
Evidence catalog	"Lab specimen" category still present	Still a real, selectable filter in the UI.

Area	Status	How I know
Blockchain & Cybersecurity theme fit	Already true, not said out loud	The custody log is an append-only, hash-chained record — that already is the "tamper-evident ledger" the theme is asking for. Nothing in the report or pitch currently says this.
BSA Section 63 certificate format	Missing	reporting.py has no wording tied to Section 63 or a named responsible official — the report isn't shaped like the specific legal certificate format Indian courts look for.
Clock-drift warning	Missing	drift_offset_seconds silently defaults to 0 when nobody's calibrated it. No warning is shown when a device's clock was never checked.

4. Who does what

Split by real effort, not how many bullet points each person has — one hard reverse-engineering bug can take longer than five small fixes put together. If anyone's list turns out heavier than it looks once they're in it, say so — the two smallest, most self-contained items on Jai Pranav's list (removing the Settings panel, removing the Evidence catalog category) can go to whoever finishes their own list first. That's the release valve; use it instead of quietly falling behind.

Every item below ends the same way: working on real evidence, demonstrated live, not shown as a recording or described as "should work."

Navneet — Cross-camera trace (own scope)

- Get real multi-**angle** footage, not just multi-time-window footage. The current demo proves the matching pipeline (build short tracks -> group them -> search) works across separate sources, but the two "cameras" used were two time windows of one real camera feed, not two different physical viewpoints. Matching someone's appearance from a genuinely different angle is the harder and more honest test — even two phones filming the same walk from different sides is enough to test it properly.
- Optional, and only after checking with Roshan: the problem statement's own phrase is "correlate events across cameras," which is broader than matching one person. Lining up Findings' motion/object timestamps across channels on Timeline's shared axis would answer that more completely. Don't start this if Roshan's shared-axis work already covers it — no point building the same thing twice.
- Make sure the demo actually shows the evidence trail: click a detection -> open the frame -> save it -> watch the hash appear. It already works end to end; it just needs to be part of the rehearsed demo, not left for someone to discover by clicking around.
- Show a plain verdict next to each cross-camera match, not just a raw similarity number. The similarity score and the time gap between the two sightings are already computed — the only new work is a simple rule on top: call it a strong match when the similarity is high and the time gap is realistic for someone walking between two cameras, and flag it for manual review otherwise. A bare number makes an investigator do the judgment call themselves; a stated verdict with its reasoning attached is more defensible and matches how similar tools present this kind of match.
- **Done when:** every cross-camera match shown in the UI carries a verdict and the reasoning behind it, not just a number.

Aravind — Hikvision recovery

- Find out why lab_hikvision_fs.img identifies correctly but recovers zero recordings. Start in HikvisionAdapter._scan / list_recordings — identification reads the same index recovery is supposed to read, so something in between is dropping every entry.
- If a real (not lab-built) Hikvision disk image can be found, use it instead and re-check every byte offset in docs/reference/hikvision_fs.md against it.
- **Done when:** the lab image, or a real one, recovers a correct, non-zero number of recordings, with at least one genuinely marked deleted, and a test that checks this automatically.

Aslam — Dahua filesystem and the video-repackaging step

- Replace the current DHAV byte-scanning with real DHFS index parsing (the index starts with a DHFS4.1 marker) — read the recording list from the index the same way Hikvision's parser reads HIKBTREE, instead of scanning the whole disk for a repeating signature. Keep the current byte-scan as a documented fallback for when the index itself is damaged.
- A recording is "deleted" when its byte range falls outside anything the index currently claims — replace the current guess (any 4KB gap between scanned frames) with that real check.
- The ranged-export fix from the last audit round was verified on Hikvision-style data; re-check it specifically against a Dahua recording, since that's a different code path.
- **Done when:** Dahua recovery reports a deleted count that came from reading the index, not from guessing at gaps.

Roshan — Timeline and playback across channels

- Check that grouping recordings by channel in timeline.py actually produces one shared time axis in TimelineView.tsx, where moving the playback position on one channel moves all of them together. This was flagged as unclear in the last audit — confirm it's actually wired through, not just present on the backend with nothing using it on the frontend.
- Deleted recordings need a visibly different color on the timeline, a legend explaining it, and a toggle to show only deleted ones — driven by the real deleted/not-deleted state Aravind and Aslam's parsers now report, not a placeholder.
- Small addition: show a visible warning on any device whose clock was never calibrated, instead of silently treating its drift as zero. A recorder clock that's never been checked against real time is a real forensic problem, and it needs to be visible, not assumed away.
- **Done when:** a case with two channels shows both on one shared timeline, deleted recordings look different from normal ones, moving one channel's playback moves both, and an uncalibrated device says so on screen.

Jai Pranav — Report, Job log, Settings, Evidence catalog, and pulling it all together

- **Report:** change the generated report's colors off the dark background (#0a0e1a -> white background, near-black text, one blue accent) to match every other screen. This is the last place the rejected dark theme is still showing up.
- Small addition: shape the report's signing section like a real **BSA Section 63 certificate** — device details, hashes, how the evidence was extracted, and who did it, all filled in and ready for a named person to sign. The tool fills in the form; a human still has to sign it — that's the correct way to describe it, and it's the one specific legal document this problem statement implies that nothing in the app currently produces.
- **Job log:** add a column showing what kind of job each row is, and stop hiding everything except recovery jobs — analytics, export, and cross-camera jobs need to show up here too.
- **Settings:** remove the "Parser sanity check" panel and the backend behind it — it's a test suite dressed up as a product feature. Real tests belong in the automated test suite, not a button in the app.
- **Evidence catalog:** remove the "Lab specimen" category and its icon — it's a leftover from earlier synthetic-data testing and shouldn't be a real option in a finished tool.
- **Pulling it together:** update or retire scripts/demo/mvp.sh — it still has a step for Live devices, which no longer exists. Then run the whole demo script, start to finish, on the real Dahua .dav, and confirm every step from Section 1 actually happens on screen, not just in theory.
- **Done when:** the report is light-themed and shaped like a real Section 63 certificate, the job log shows every kind of job, Settings and Evidence catalog have nothing fake left in them, and the demo script runs clean from a fresh case to a signed report.

5. How this actually gets judged

A 3x official SIH evaluator's own advice, in plain terms: AI coding tools mean anyone can put together a working-looking screen in an afternoon, so just having a demo run isn't worth much anymore — evaluators weight it at roughly 20–30% of the score now, not the 60% it used to be. The rest is architecture, whether the code is actually organized sensibly, and whether the person presenting a piece of it can explain how it works, not just click through it. In practice: everyone here should be able to explain their own part from memory, not read it off a slide.

Five questions worth having a real answer ready for:

1. Is your input data real or synthetic? — Real Dahua .dav, a real Hikvision NVR export, real Digital Corpora .E01 disk images. Synthetic only where it's a filesystem-structure fixture built for testing, and that gets said plainly, not hidden.
2. What's the AI actually doing that a simpler rule-based system couldn't? — Answer per feature, not in general. Matching a person's appearance across cameras genuinely needs a learned model — there's no rule for that. Motion and scene-change detection are plain image comparison, not AI. The proximity flag is just checking whether two boxes overlap. Don't call the non-AI parts AI.
3. What happens when something's missing or fails? — It says so. A model that isn't installed skips with a clear message instead of crashing. Damaged recordings get marked partial instead of silently dropped. Confidence gets stated, not assumed.
4. Would this hold up on real production-scale data? — Honestly: it's been checked against real individual devices, not a fleet of them. Say that directly instead of overselling it.
5. How do you know the hash record itself wasn't changed after the fact? — Each custody entry's hash depends on the one before it, so changing an old entry breaks every entry that comes after it — that's what "append-only, hash-chained" means in practice. This is also a direct answer to the "Blockchain & Cybersecurity" theme this problem statement is filed under, and it should be said outright in the report and the pitch, not left implicit.

On claiming support for all 8 vendors: don't. The problem statement itself says teams realistically go deep on one or two vendors, not all eight — which is already the shape of what's built here (Dahua, Hikvision, and Honeywell parsed properly; the rest get acquisition and a generic recovery attempt, clearly labeled as such). Saying that plainly up front is a stronger position than a claim that falls apart the first time someone asks to see an eighth vendor live.

The presentation slides follow the standard SIH format: Problem Understanding, Tech Stack & Architecture, Feasibility/Risks & Mitigation, Real-World Impact, Research & References. The architecture slide needs to show real backend structure, not a screenshot of the UI. The references slide should cite the same sources already backing the Hikvision engine (a 2015 forensics conference paper, a 2023 Journal of Forensic Sciences paper, a 2025 paper in the journal Information) — they're already written down in docs/reference/hikvision_fs.md; they just need to be copied onto a slide.

One note on the AI models, in case other material on this problem statement suggests otherwise: Pramaan runs small, CPU-only models (YOLOX-nano for detection, a compact re-identification model, all through OpenCV, no PyTorch), on purpose — it's a deliberate choice to keep compute cost and install size low, because judges take that seriously. If you've seen a plan for this problem statement that uses a heavier stack (like YOLOv11 with a full tracking and face-recognition pipeline), that's a different team's approach, not this one — don't switch to it.

What other public teams have built for this exact problem statement: checked the four other public GitHub repos for SIH26150. One (MIT-licensed, called "24fps") is a real, disciplined build — genuine byte-level parsing for one vendor, real ML inference, deterministic multi-signal cross-camera correlation (which is where the verdict idea above came from), and reproducible PDF reports. Nothing was copied from it — the ideas above were rebuilt independently in Pramaan's own code and style. The other three aren't real competing attempts: one's actual code solves an unrelated hackathon problem with the title swapped, one is mostly empty folders with an admitted "no active forensic algorithms yet," and one doesn't even start up (a broken import in its only backend service). None of the four have any real DVR/NVR sample footage or disk image sitting in the repo that we could use for testing.

6. Before calling anything done

- `python -m pytest engine/tests -q` passes.
- Frontend typecheck, build, and format check all pass on whatever you changed (`npm run build`, `npx tsc`, `npm run format:check`).
- The "done when" line for your section, shown working on real evidence — not a fixture, not on the first attempt, not described as "should work."
- Nothing you touched leaves a placeholder, a fake number, or a feature that only works in one specific case behind. If it's not really done, it's not in the demo.